

Informe Técnico

Etiquetador de Imágenes de Moda

Asignatura	Visión por Computador — Práctica de Recuperación de Imágenes
Tecnología	Python 3 · NumPy · Matplotlib
Fecha	03 de May de 2026
Versión	1.0 — Revisión final

Índice de Contenidos

1.	Visión General	3
2.	Arquitectura por Archivos	3
2.1	Archivos del curso (utilidades)	3
2.2	KNN.py — Clasificador de forma	4
2.3	Kmeans.py — Extracción de color	4
2.4	my_labeling.py — Núcleo lógico	5
2.5	main.py — Interfaz de usuario	5
3.	Flujo de una Consulta	5
4.	Decisiones de Diseño	6
5.	Cómo se Ejecuta	6
6.	Mejoras Propuestas	7

1 Visión General

Sistema de recuperación de imágenes de moda por consulta en lenguaje natural. El usuario especifica tipo de prenda y color en inglés; el sistema etiqueta automáticamente imágenes del conjunto de test combinando un clasificador KNN (forma) y agrupación K-means (colores dominantes en RGB), y devuelve los índices de las imágenes que coinciden con la consulta.

Componente	Tecnología	Rol principal
Clasificación de forma	KNN (k=5, distancia euclídea)	Predice tipo de prenda
Extracción de color	K-means sobre RGB	Identifica colores dominantes
Interfaz	Menú de texto (main.py)	Entrada/salida para el usuario
Utilidades	utils.py / utils_data.py	Proporcionadas por el curso

2 Arquitectura por Archivos

2.1 Archivos del curso (utilidades)

Archivo	Rol	Exportaciones clave
utils.py	Utilidades de visión: conversión de color, modelo Lab2RGB y RGB2Lab	rgb2gray, rgb2color_prob, colors, RGB2Lab
utils_data.py	Carga del dataset, visualización de resultados	read_dataset, visualize_retrieval

2.2 KNN.py — Clasificador de forma

Responsabilidad: clasifica el tipo de prenda de una imagen usando distancia euclídea sobre vectores de escala de grises.

Elemento	Entrada	Salida
read_dataset(gt_json, **kw)	Ruta JSON del ground-truth	Tupla de splits (train/test/labels)
_vec(im) (privada)	Imagen RGB o gris	Vector 1-D float64 (convierte a gris si procede)
KNN(X, y, k=5) __init__	Imágenes train + etiquetas	Modelo: vectoriza en construcción
KNN.predict(T)	Lista de imágenes	Array de etiquetas predichas (voto mayoritario)

Dependencias: utils.rgb2gray (conversión a gris), utils_data.read_dataset (carga del dataset).

2.3 Kmeans.py — Extracción de color

Responsabilidad: agrupa los píxeles de una imagen en K clusters RGB e identifica el nombre de color de cada centroide.

Elemento	Entrada	Salida
imagen_a_puntos_rgb(im)	Array H×W×3	(H·W)×3 float64

Elemento	Entrada	Salida
Kmeans(X, K=3) __init__	Nube de puntos + K	Objeto con X, K, n, centroids, labels
Kmeans.fit(max_iter, semilla)	Parámetros opcionales	self (actualiza centroids y labels)
Kmeans.withinClassDistance()	—	Float: suma cuadrados intra-cluster (WCD)
Kmeans.find_bestK(max_K=8)	K máximo a explorar	Int: K óptimo (mejora relativa < 20 %)
Kmeans.get_color()	—	Lista de strings de color (uno por centroide)

Dependencias: utils.colors (tabla de nombres), utils.get_color_prob (asignación en espacio Lab).

2.4 my_labeling.py — Núcleo lógico

Responsabilidad: parsea la consulta del usuario, orquesta KNN y K-means por imagen y expone las funciones de búsqueda y evaluación de precisión.

Función / constante	Entrada	Salida
AYUDA (constante)	—	Cadena de ayuda visible en el menú
parsear_consulta(t)	String EN	(tipo, color) o None si no reconoce ambos
pprint_hits(ix, lim=15)	Lista de índices	Imprime en consola (trunca a lim)
colores_predominantes(im, max_K, salto)	Imagen	Lista de strings de color dominante
etiquetar_imagen(im, knn, max_K, salto)	Imagen + modelo KNN	(tipo_predicho, lista_colores)
buscar(consulta, ...)	Consulta + parámetros	(índices, imágenes_test, knn, (tipo, color))
precision_busqueda(consulta, ...)	Consulta + parámetros	(precisión, n_candidatos, tipo) o None
_pruebas_automaticas()	—	Smoke-tests con asserts; imprime OK o lanza AssertionError

Dependencias: KNN (clasificación), Kmeans (colores), utils.colors (lista de colores válidos).

2.5 main.py — Interfaz de usuario

Responsabilidad: menú interactivo de texto; no contiene lógica de negocio propia.

Elemento	Responsabilidad
main()	Bucle de menú (opciones 0–4): delega en my_labeling y utils_data
num(s, default)	Parsea entero del usuario; acepta vacío o 'todas' como default
si(s)	Interpreta confirmación S/N del usuario

Dependencias: my_labeling (todo el núcleo), utils_data.visualize_retrieval, numpy.

3 Flujo de una Consulta

Ejemplo con la consulta "Black shorts", opción 1 del menú:

Paso	Módulo	Operación
1	main.py	Usuario introduce "Black shorts" y límite N de imágenes
2	my_labeling	parsear_consulta → tipo="Shorts", color="Black"
3	utils_data	read_dataset() → (train_imgs, train_labels, ..., test_imgs, ...)
4	KNN.py	KNN(train_imgs, train_labels, k=5): vectoriza imágenes a gris 1-D en __init__
5	my_labeling	Bucle for i in range(N): etiquetar_imagen(test[i], knn)
6	KNN.predict	Distancia euclídea contra todos los vectores train → voto k=5 → tipo_predicho
7	Kmeans.py	colores_predominantes: submuestreo im[:,salto, :salto] → puntos RGB
8	Kmeans.find_bestK	Evalúa WCD para K=2..8; detiene cuando mejora relativa < 20 %
9	Kmeans.fit	Ajusta K-means con K óptimo → centroids
10	Kmeans.get_color	Mapea cada centroide a nombre de color vía get_color_prob (espacio Lab)
11	my_labeling	Filtro: si tipo_predicho=="Shorts" y "Black" in colores → añade i a ix
12	main.py	Muestra conteo + pprint_hits(ix); opcionalmente visualize_retrieval

4 Decisiones de Diseño

Representación de forma (gris + ravel):

Cada imagen se convierte a escala de grises con `rgb2gray` y se aplanan en un vector 1-D. Captura la estructura geométrica prescindiendo de la información cromática, reduciendo la dimensionalidad sin perder la forma de la prenda. El tamaño del vector depende de la resolución de carga del dataset (verificar en `utils_data`).

KNN con `k=5` fijo:

Valor por defecto razonable para datasets de tamaño moderado. No se valida con cross-validation; todos los parámetros son sobrescribibles desde `buscar()`. La elección de `k=5` equilibra varianza y sesgo sin necesidad de búsqueda de hiperparámetros.

K-means en RGB con criterio 20 %:

`find_bestK` detiene la búsqueda en el primer K donde la mejora relativa de WCD cae por debajo del 20 %. Sencillo, sin hiperparámetros adicionales. Puede infraestimar K en imágenes con colores muy distintos y baja transición de WCD.

Nombres de color vía `get_color_prob` (Lab implícito):

Aunque K-means opera en RGB, `get_color_prob` de `utils` convierte internamente a Lab (`RGB2Lab`) para asignar el nombre de color. Esto mejora la coherencia perceptual del etiquetado respecto a una comparación directa en RGB.

Submuestreo de píxeles con salto:

`im[::salto, ::salto]` reduce el número de puntos analizados por K-means. Con `salto=2` se procesa el 25 % de los píxeles, acelerando la parte más costosa del sistema sin cambiar la lógica de agrupación. Salto configurable en todas las funciones públicas.

Separación menú / `my_labeling`:

`my_labeling.py` ejecutado directamente sin `--test` solo muestra un aviso y redirige a `main.py`, evitando uso accidental como CLI de búsqueda. La suite de tests se activa explícitamente con `--test`, facilitando la validación rápida sin levantar el menú.

5 Cómo se Ejecuta

Modo	Comando	Descripción
Menú interactivo	<code>python3 main.py</code>	Lanza el menú (opciones 0–4)
Tests automáticos	<code>python3 my_labeling.py --test</code>	Ejecuta smoke-tests con asserts
Consulta de ejemplo	Black shorts (en opción 1 del menú)	Tipo + color válidos en inglés

Ejemplos de consultas válidas:

Consulta	Tipo extraído	Color extraído
Pink dress	Dresses	Pink
Black sandals	Sandals	Black
White shirts	Shirts	White
Blue jeans	Jeans	Blue

Nota: la consulta debe contener obligatoriamente un color y un tipo de prenda reconocibles en inglés. Si falta cualquiera de los dos, `parsear_consulta` devuelve `None` y el sistema lanza un aviso sin procesar imágenes.

6 Mejoras Propuestas

01

Caché del modelo KNN

El clasificador se reentrena en cada llamada a `buscar()`. Guardarlo tras la primera carga (con `functools.lru_cache` sobre `read_dataset` o persitiendo el objeto) eliminaría el cuello de botella principal en consultas repetidas.

Alto impacto · Bajo coste de implementación

02

Paralelismo en el bucle de etiquetado

El bucle `for i in range(n)` en `buscar()` es embarazosamente paralelo. `concurrent.futures.ProcessPoolExecutor` reduciría el tiempo de consulta de forma casi lineal con los núcleos disponibles del sistema.

Alto impacto · Coste medio

03

Validación de k en KNN

Añadir búsqueda de k óptimo por validación cruzada (o exponer métricas de accuracy sobre el split de test) daría rigor a la elección de `k=5` y permitiría adaptarse a distintos tamaños de dataset.

Impacto medio · Coste medio

04

Normalización de resolución de imagen

Si `utils_data` carga imágenes a distintas resoluciones, un reescalado explícito a tamaño fijo (por ejemplo `80×60`) garantizaría vectores de igual longitud y evitaría errores silenciosos en la concatenación de vectores.

Impacto medio · Bajo coste

05

Soporte multilinguaje en consultas

`parsear_consulta` solo reconoce inglés. Añadir alias en español y catalán, o integrar un paso ligero de traducción, haría el sistema coherente con el menú bilingüe ya presente en `main.py`.

Impacto bajo · Bajo coste

Documento generado automáticamente · Revisión de código sénior · 03/05/2026